

Centro Universitário do Distrito Federal (UDF)

Disciplina: Engenharia de Prompt e Aplicações em IA

Orientadora: Profa. Kadidja Valéria

Integrantes: Evellyn, Gabriel, Pedro e Guilherme.

1. INTRODUÇÃO E ESCOPO DO DESAFIO

O presente relatório documenta a concepção e a implementação de uma aplicação web voltada ao controle de finanças pessoais, projetada para mitigar o problema crônico da desorganização orçamentária individual. O software foi modelado para atender demandas dinâmicas do cotidiano, centralizando o gerenciamento de múltiplas fontes de receita e despesas.

1.1 Funcionalidades de Negócio Mapeadas

- **Gestão Dinâmica de Receitas:** Entrada unificada de valores salariais fixos e integração de campos flexíveis para o controle de receitas variáveis (ex: ganhos oriundos de atividades de motorista de aplicativo/Uber).
- **Módulo Avançado de Crédito:** Controle individualizado de cartões de crédito, permitindo parametrizar o nome da instituição, limite operacional, dia de fechamento e dia de vencimento da fatura.
- **Controle de Fluxo Temporal:** Separação entre "Modo Realizado" (dinheiro efetivamente movimentado em conta) e "Modo Previsto" (projeções orçamentárias baseadas em despesas pendentes e receitas estimadas).

2. METODOLOGIA E PLATAFORMA UTILIZADA

A equipe adotou uma estratégia de desenvolvimento puramente baseada no ecossistema **No Code** através da plataforma **Antigravity** (Versão Paga). Toda a geração de código estrutural, estilização e lógicas reativas foram delegadas aos múltiplos modelos de Inteligência Artificial encapsulados na ferramenta, eximindo o grupo de codificação manual direta em linguagens de programação.

3. ARQUITETURA TÉCNICA E ENGENHARIA DE SOFTWARE

O mapeamento reverso do código gerado revelou que a aplicação ultrapassou o escopo de um site estático comum, consolidando-se como uma estrutura de software robusta baseada em quatro pilares.

Plaintext

```
/ProjetoModulo3
├── README.md
├── /docs
│   ├── prototipo.png (Captura com CSS de impressão nativo)
│   └── relatorio.pdf
```

3.1 Pilares Estruturais da Solução

1. **Ecosistema PWA (Progressive Web App):** A solução foi implementada utilizando um manifesto de aplicação (manifest.json) e um Service Worker ativo (sw.js). Essa infraestrutura garante capacidades *Offline-First*, permitindo o funcionamento pleno da lógica de dados mesmo sem conexão ativa com a internet.
2. **Camada de Persistência Transacional (IndexedDB):** Para evitar a volatilidade de estados em memória, foi configurado o banco de dados transacional local **IndexedDB** estruturado em 6 Object Stores fundamentais: transactions, categories, accounts, cards, fixed_expenses e fixed_income.
3. **Lógica Monetária e Escalar:** Para mitigar falhas nativas de arredondamento de ponto flutuante comuns no interpretador JavaScript, o motor financeiro opera estritamente com **valores em centavos** (números inteiros) em nível de banco de dados, sendo formatados na interface através da API nativa Intl.NumberFormat.
4. **Isolamento de Complexidade (FinanceCore):** Como o arquivo principal index.html escalou para um monolito de aproximadamente **2400 linhas**, a complexidade ciclomática foi blindada isolando as funções de cálculo, formatação e aplicação de máscaras de entrada em um namespace global denominado FinanceCore.

4. ENGENHARIA DE PROMPT E ESTRATÉGIA ANTI-DECAIMENTO (OOOE)

Durante o ciclo de desenvolvimento iterativo, o principal desafio técnico enfrentado foi a regressão de código causada pelas respostas da IA (efeito colateral onde novas funções quebravam recursos previamente estabilizados).

4.1 A Técnica do "Prompt dos Prompts"

Para contornar o decaimento e os bugs gerados pela plataforma, a equipe utilizou um meta-prompt avançado com lógica iterativa de auto-correção, estruturado a partir da perspectiva do usuário fazendo requisições a um agente especialista em Engenharia de Prompts.

A estratégia consistiu em forçar a IA a se auto-avaliar dividindo sua saída obrigatoriamente em três blocos: **Prompt Otimizado**, **Crítica Construtiva Severa** (suposições e problemas de escopo) e **Perguntas de Esclarecimento** (máximo de 5 questionamentos direcionados para refinar o contexto).

4.2 Aplicação do Protocolo Guard-Rail via OODE

Para implementar correções complexas — como o sincronismo de parcelamentos futuros vinculados a servidores de tempo externo (worldtimeapi.org) —, o grupo utilizou a técnica **OODE** nos comandos. Foi determinado que a IA realizasse um loop reverso obrigatório antes de cada entrega de código, validando os seguintes *Guard-Rails* arquiteturais:

- **Data Integrity First:** Bloqueio de inserções sem validação prévia de esquema.
- **Read-Only Calculations:** Clonagem obrigatória de objetos antes de realizar mutações de saldo.
- **UI State Decoupling:** Separação entre o estado dos dados e os elementos visuais do DOM (utilização de data-attributes para vinculação segura de IDs).

4.3 Restrições e Limitações Identificadas da Plataforma

Apesar da alta eficiência no tempo de entrega da solução, o desenvolvimento puramente baseado no ecossistema No Code da plataforma Antigravity revelou barreiras técnicas claras. Conforme exigido pelos critérios de avaliação do projeto, foram identificadas as seguintes limitações:

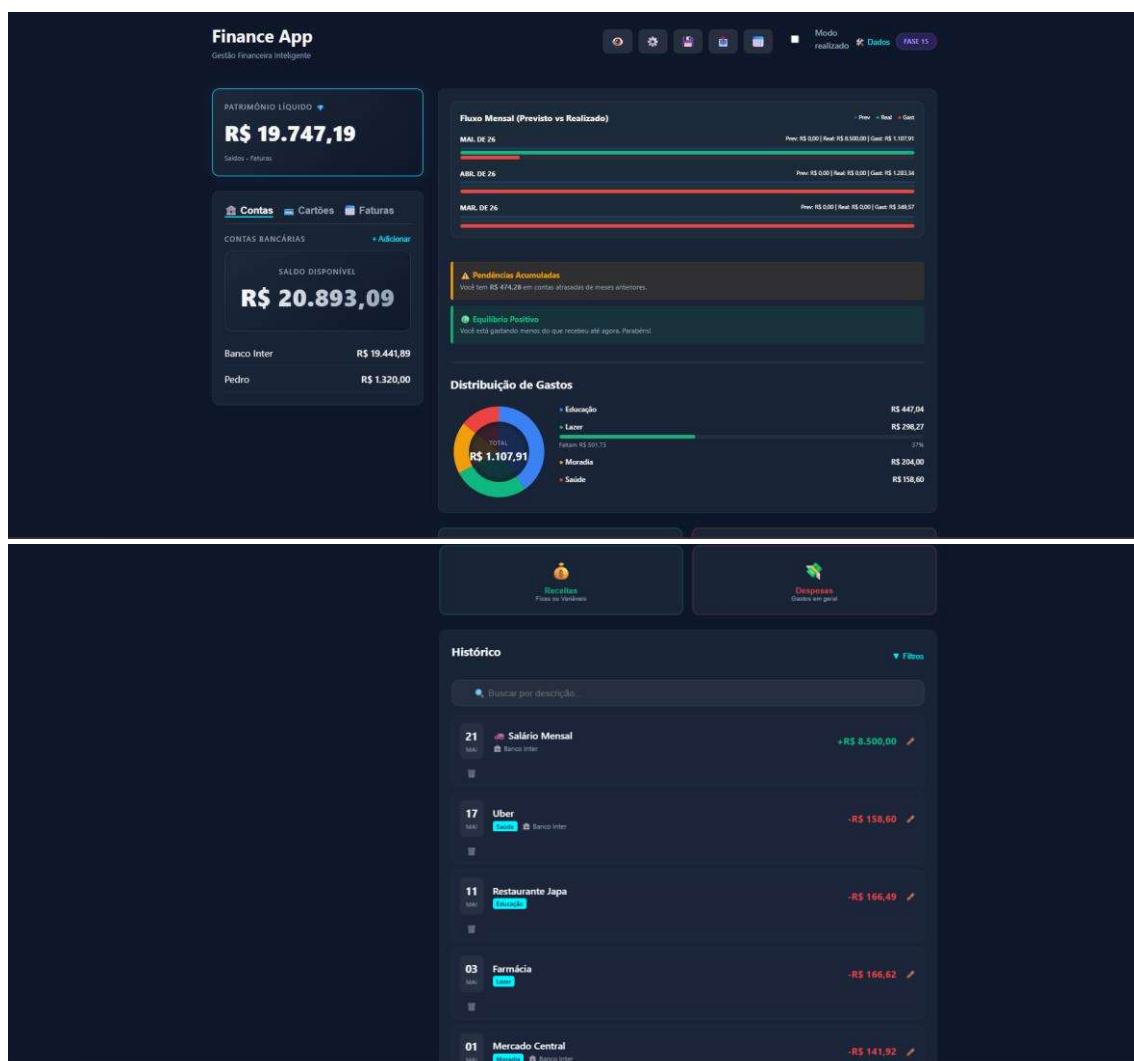
1. Efeito de Decaimento e Regressão de Código (Instabilidade): Atualizações incrementais de código geradas pela Inteligência Artificial frequentemente causavam efeitos colaterais severos, onde a implementação de novos recursos quebrava lógicas e funcionalidades que já haviam sido previamente estabilizadas e testadas pela equipe.
2. Tendência à Monolitização Extrema (Complexidade Ciclomática): A plataforma demonstrou limitação no gerenciamento e na modularização

nativa dos arquivos de código, tendendo a concentrar toda a estrutura lógica, estilização e comportamento em um único arquivo estrutural (index.html com aproximadamente 2400 linhas), elevando a complexidade e demandando esforço manual futuro para refatoração.

3. Dependência de Conexão e Serviços Externos para Sincronismo: A ferramenta apresentou restrições para lidar de forma nativa e autônoma com o sincronismo temporal confiável de parcelamentos futuros. Isso obrigou o grupo a introduzir dependências de APIs de terceiros (como a *worldtimeapi.org*), limitando a autonomia e a independência tecnológica da aplicação local.

5. DOCUMENTAÇÃO VISUAL E COMPORTAMENTO DE INTERFACE

As evidências visuais capturadas pelo grupo demonstram a reatividade do sistema e a versatilidade de customização construída pela IA.



5.1 Interfaces e Temas Implementados

O painel do sistema conta com três folhas de estilo intercambiáveis configuradas via variáveis nativas do CSS (:root), permitindo alternar a experiência do usuário em tempo real entre o tema padrão **Dark Blue**, o tema estilizado **Nubank** (predominância de tons roxos) e o tema **Matrix** (estética hacker em tons verdes e pretos).

5.2 Nota Técnica sobre o Arquivo prototipo.png

Conforme documentado na pasta /docs, o arquivo de imagem prototipo.png apresenta-se com o fundo em coloração clara (branca). Esta característica deve-se à captura de tela ter sido gerada via comando nativo de impressão do navegador (Ctrl + P), disparando as folhas de estilo de mídia de impressão (@media print) programadas no navegador, as quais convertem automaticamente o espectro escuro para uma paleta clara e de alto contraste visando a economia de insumos, comprovando o comportamento padrão e responsivo do código gerado.

6. COLABORAÇÃO

A equipe dividiu a execução das tarefas e responsabilidades de forma estratégica para maximizar a eficiência do fluxo No Code[cite: 1]:

- Evellyn: Responsável pela concepção da ideia do projeto, engenharia inicial de prompts e publicação/gerenciamento do repositório no GitHub.
- Gabriel: Responsável pela análise técnica, refinamento e refatoração dos prompts para correção de bugs e aplicação da metodologia OODE.
- Pedro: Responsável pela confecção, estruturação e revisão dos documentos acadêmicos do projeto.
- Guilherme: Responsável pela confecção, estruturação e mapeamento visual dos documentos contidos na pasta

7. CONCLUSÃO E PRÓXIMOS PASSOS

A abordagem No Code demonstrou-se altamente eficaz para o desenvolvimento acelerado de soluções viáveis (*Time-to-Market*). O projeto provou que o uso assistido de IAs generativas permite a estudantes modelarem softwares com

recursos arquiteturais complexos (como PWAs e IndexedDB) sem barreiras de sintaxe técnica.

Como evolução futura mapeada para o encerramento do Projeto Final da Unidade 3, o grupo planeja desacoplar as 2400 linhas do monolito index.html em módulos independentes JavaScript (ES Modules), adicionar um sistema de criptografia local para proteção dos dados do IndexedDB e expandir o painel gráfico de previsão analítica de faturas futuras.